

Canonical Rendering Sprint — the manual builder becomes a renderer over the scenario graph

Author: QA Architecture

Branch: feat/canonical-manual-renderer

Status: Ready for review — proven end-to-end on ONE capability (Sorting), zero regression elsewhere.

1. The problem this sprint fixes

The manual Test-Case-Lab output and the automation Script-Gen output are supposed to be two views of the same test. They were not. They were produced by **two independent generators** that could — and did — disagree.

The clearest evidence was the 20-case sorting export (`test-cases-238.xlsx` , REQ-005), which scored 2/10: every “Sort products” case rendered **checkout steps** (“Open checkout → fill First/Last/Zip → click Continue”) and a **CRUD expected result** (“The Sort products record is created successfully”). The requirement was about sorting; the output was about a form the app happened to expose.

Root cause (code-grounded, not guessed)

The scenario **graph** is already the single source of truth. It is consumed by **two** adapters:

Adapter	Consumer	What it read
<code>toScriptGenSpecs()</code>	Automation / Script Gen	<code>node.actions</code> + <code>node.assertions</code> (the canonical machine model)
<code>toTestCaseLab()</code>	Manual test cases	<code>node.steps</code> — legacy prose the old form-playbook builder guessed

So Script Gen rendered from the canonical actions, while the manual side copied whatever prose the legacy `pickForm` + CRUD-template path had produced. When a capability had no authored canonical actions, the legacy path fell back to “find any form on the app and fill it” — which is exactly how a sort scenario inherited checkout steps.

The architecture was ~80% built and stopped halfway. Only `authentication` had authored `semantics` / `actionTemplate` / `assertionTemplate` . Every other category (search, checkout, crud, payment, ...) had **zero** authored canonical content, so the manual side never had canonical actions to render and always fell back to the guessing path.

2. What this sprint does — complete the contract, don't add a layer

No new abstraction was introduced. We finished the design that already exists.

2.1 Phase 1 — the manual builder renders from canonical actions

`toTestCaseLab()` now renders each column from the canonical model **when it exists**, and falls back to the exact legacy output when it does not:

```
const manual = renderManualIfCanonical(n); // null when n.actions is empty → legacy
steps        : manual ? manual.steps      : n.steps.slice()
expectedResult : manual ? manual.expectedResult : n.expectedResult
testData      : manual ? manual.testData    : composeResolvedTestData(n)
```

The projection lives in a new **pure** module `src/graph/canonical-manual-renderer.ts` (`renderManualFromCanonical(actions, assertions, resolved?)`). It humanises canonical targets app-neutrally (`sort_dropdown` → “sort dropdown”), prefers authored human text when present, and degrades deterministically otherwise. It fabricates no locators.

2.2 Completing the canonical object's human face

The FROZEN machine vocabulary could not, by itself, express a business outcome like “products are in alphabetical order” — there is no `ordered AssertionType`, and there never should be one just for prose. Two **optional, additive** fields were added so the single canonical object can carry both faces without a parallel model:

- `ScenarioAction.description?` — the human sentence for a step.
- `ScenarioAssertion.observable?` — the human-observable business outcome for a check.

Both are optional → every existing object is byte-for-byte unchanged → zero regression. They are threaded verbatim through `materializeActionTemplate` / `materializeAssertionTemplate`.

2.3 Phase 2 — author ONE capability: Sorting

`search-pos-sort` is now fully authored in the KB (`search catalog`):

```
actionTemplate:
  navigate product_list      description: "Open the product list page"
  select  sort_dropdown = "Name (A to Z)"
                                description: 'In the Sort dropdown, select the "Name
(A to Z)" option'
assertionTemplate (all afterAction = search-pos-sort.select.sort_dropdown):
  visible product_list      observable: list re-orders A→Z, matches the selected
option
  value  sort_dropdown = "Name (A to Z)"  observable: selected option retained after
re-order
  visible product_item (opt.)  observable: each product's name/image/price unchanged
  visible cart (opt.)          observable: cart contents & count preserved
```

Note the cart-preserved and product-unchanged invariants ride on the **authored KB assertion**, independent of how the requirement happens to be worded.

3. Proof — before vs after, from the REAL pipeline

`scripts/canonical-render-proof.ts` runs the exact engine path
 (`planScenarios` → `buildDraftTestCases` → `buildDeterministicOutput` → `materialize templates` → `assembleScenarioGraph` → `toTestCaseLab`). The App Profile deliberately exposes **only a checkout form**, reproducing the original bug's conditions.

BEFORE (legacy `node.steps` — the 2/10 artifact):

```
1. Open the page under test
2. Enter "Sophia" in the First Name field
3. Enter "Johnson" in the Last Name field
4. Enter "Sample Zip/Postal Code" in the Zip/Postal Code field
5. Click the Continue button
Expected: ✓ The Sort products record is created successfully. | ✓ A success
confirmation ... | ...
```

AFTER (`canonical-rendered toTestCaseLab`):

```
1. Open the product list page
2. In the Sort dropdown, select the "Name (A to Z)" option
Expected Result:
  ✓ The product list re-orders into ascending alphabetical order by name (A → Z),
  matching the selected option.
  ✓ The Sort dropdown still shows "Name (A to Z)" — the selected sort option is re-
  tained after the list re-orders.
  ✓ Each product's name, image and price are unchanged — only the display order
  changed, not the products themselves.
  ✓ The shopping cart contents and item count are preserved — sorting the list does
  not modify the cart.
Test Data: sort dropdown: Name (A to Z)
```

Zero-regression check (same run): a non-authored scenario (`search-pos-match`) is
 byte-identical between `node` and projection — `steps unchanged=true`, `expected unchanged=true` .

4. Validation

- `npx tsc --noEmit` — **clean** (exit 0).
- Proof harness — reproduces BEFORE (broken) and AFTER (correct) plus the zero-regression check.
- Graph / adapter / builder / planner / renderer suites — all green
 (`scenario-graph` 22, `scenario-graph-adapters` 18, `scenario-graph-provider` 12,
`scenario-renderer` 14, `scenario-builder` 28, `scenario-planner` 53, `scenario-correctness` 25,
`scenario-integrity` 32, `scenario-specific-test-data` 16, `canonical-test-case` 35,
`canonical-test-data` 33, `script-gen-scenario-fidelity` 29 ...).
- KB invariants: `qa-knowledge-actions-invariant` / `qa-knowledge-assertions-invariant` — the
`afterAction-links-a-real-action-id` invariant (**F**) passes for the sort scenario, confirming
`search-pos-sort.select.sort_dropdown` matches its materialized action id.

Deliberate ratchet update (reviewed, not a silent break)

Sprint 2D installed a guard: “no non-authentication module may author action/assertion templates until auth is 100%.” This sprint deliberately authors ONE search-category scenario, so both

guards were updated with an **explicit, commented allow-list** of a single entry (`search/search-pos-sort`) rather than being weakened. Everything else the guards protect stays enforced.

Pre-existing failures NOT caused by this sprint (disclosed)

On `main`, three KB-invariant assertions already fail because the `auth-reset-*` scenarios were shipped without authored templates (auth is not actually at 100%). These are unrelated to this sprint and were red before any change here. Left as-is; flagged for a future auth-completion task.

5. Honest gap — automated ordering has no assertion primitive yet

The manual view tells the full truth via `observable` (“re-orders into ascending alphabetical order”). The machine view cannot yet **verify ordering**: the frozen `AssertionType` vocabulary has no `ordered` primitive, so Script Gen currently asserts the closest checkable property (`visible product_list`) and the ordering intent lives only in the human `observable`.

This is a clearly-scoped follow-up: **add an ordering assertion primitive** so Script Gen can verify the sequence, not just the presence, of the list. This is internal engineering backlog — **not** a customer-facing placeholder or TODO. Nothing degraded ships to a customer.

6. Scope boundaries (as agreed)

- **Classification is out of scope.** “Sort products” still classifies under `search` (there is no `sorting` category), and the classifier is keyword-scored — the word “order” in a requirement can pull it toward `checkout`. We did **not** touch classification code this sprint.
- **Script generation is untouched.** Only the manual renderer changed. Both adapters still read the one graph.
- **PR #320 (`requirement-capability-detector`) stays parked.** This branch is based on `main`, not on that branch.

7. What Phase 4 becomes: authoring, not engineering

The engine work is done. Adding the next capability (Search, Filter, Upload, CRUD, Approval, Checkout, Payment ...) is now purely **authoring** a KB entry — `semantics` + `actionTemplate` + `assertionTemplate` with `description` / `observable`. Both the manual and script views render it automatically from the one source. Each new capability should be authored, reviewed on ONE scenario end-to-end (as done here), then expanded.

8. Files changed

File	Change
src/graph/scenario-graph.ts	+optional ScenarioAction.description , ScenarioAssertion.observable
src/engines/qa-knowledge-engine.ts	+template fields; authored search-pos-sort (Sorting)
src/graph/scenario-graph-builder.ts	thread description / observable through materializers
src/graph/canonical-manual-renderer.ts	NEW pure projection: canonical actions/ assertions → manual columns
src/graph/scenario-graph-adapters.ts	toTestCaseLab() renders from canonical when present; legacy fallback
tests/unit/qa-knowledge-actions-invariant.test.ts	ratchet allow-list (reviewed)
tests/unit/qa-knowledge-assertions-invariant.test.ts	ratchet allow-list (reviewed)
scripts/canonical-render-proof.ts	NEW before/after + zero-regression proof harness